

VISUAL GUIDE

Tensors & CUDA

Understanding the data structure at the heart of PyTorch and TensorFlow — and how the GPU, through CUDA, turns tensor math into massive parallel speed.

What a tensor really is

PyTorch & TensorFlow

GPU parallelism

CUDA execution model

Mixed precision & Tensor Cores

15 diagrams

A concept-first reference with hand-drawn figures · mid-2026 · covers CUDA Toolkit 13.x, cuDNN 9.x, PyTorch 2.x, TensorFlow 2.x

Contents

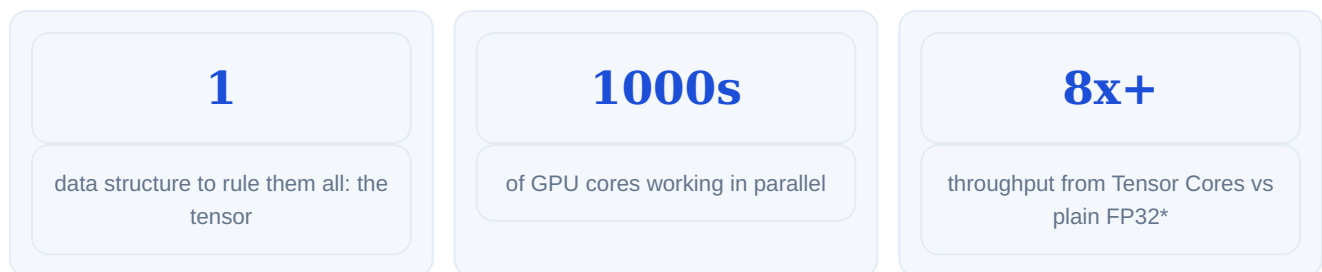
- 01** Introduction — why tensors and why GPUs
- 02** What is a tensor?
- 03** Tensors in the real world
- 04** Anatomy of a tensor: shape, rank, dtype, device
- 05** How tensors live in memory
- 06** Broadcasting
- 07** Core tensor operations
- 08** Tensors in PyTorch
- 09** Tensors in TensorFlow
- 10** PyTorch vs TensorFlow at a glance
- 11** Why GPUs are built for tensors
- 12** CUDA in a nutshell
- 13** How CUDA accelerates tensor math
- 14** Data movement & the GPU training loop
- 15** The full stack, and key takeaways
- 16** Glossary

1 Introduction — why tensors and why GPUs

Modern machine learning is, underneath all the jargon, an enormous amount of arithmetic on arrays of numbers. The **tensor** is the container for those numbers, and the **GPU** — programmed through NVIDIA's **CUDA** platform — is the engine that crunches them.

Every model you train in PyTorch or TensorFlow represents its inputs, its weights, its intermediate activations, and its gradients as tensors. Every operation — a matrix multiply in a linear layer, a convolution over an image, an attention score — is a transformation from tensors to tensors. If you understand tensors well, the frameworks stop feeling like magic.

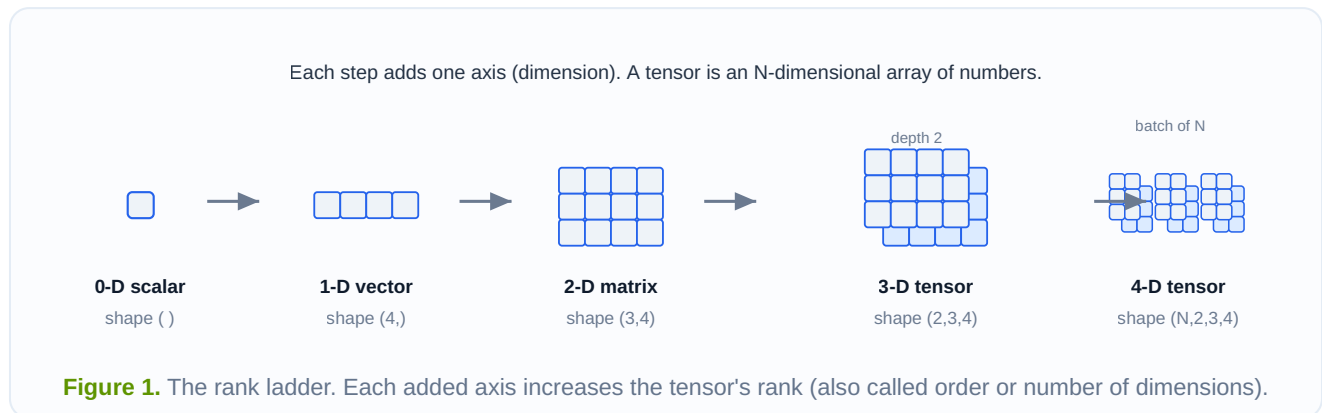
The second half of the story is speed. A single training step can involve billions of multiply-add operations. CPUs execute these mostly one after another; GPUs execute thousands at once. This guide builds up both ideas from scratch: first what a tensor *is* and how the two big frameworks handle it, then how CUDA maps tensor math onto GPU hardware to make it fast.



*NVIDIA reports FP16 matrix ops on Tensor Cores delivering roughly an 8x increase in compute throughput over standard FP32 arithmetic; exact gains vary by GPU and workload.

2 What is a tensor?

A tensor is simply an **N-dimensional array of numbers**. It generalises the familiar objects of school maths: a single number, a list, and a table are all just tensors of different dimensionality.

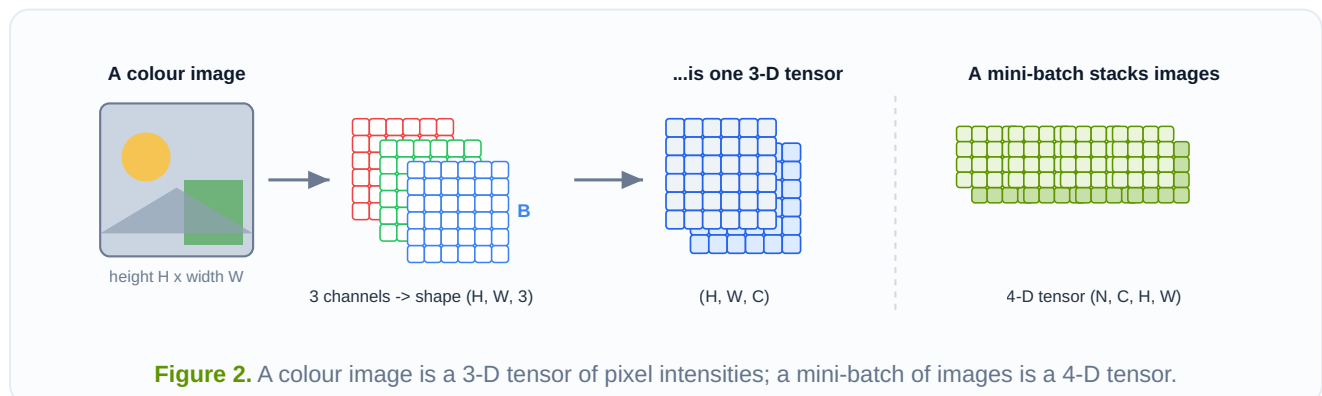


The number of axes is the tensor's **rank** (or **ndim**). A plain number is a rank-0 tensor (a *scalar*); a list of numbers is rank-1 (a *vector*); a table is rank-2 (a *matrix*); and stacking tables gives rank-3 and beyond. There is nothing mysterious past three dimensions — a 4-D tensor is just a list of 3-D tensors, and so on.

Two tensors of the same rank can still be very different in size. That size is captured by the **shape**: a tuple giving the length along each axis. A tensor of shape `(2, 3, 4)` has $2 \times 3 \times 4 = 24$ elements arranged in a little 3-D box.

3 Tensors in the real world

Tensors are how real data becomes numbers a network can process. The dimensions carry meaning.



A grayscale image is a 2-D tensor of pixel brightness values (height $\times$ width). A colour image adds a third axis for the red, green and blue channels, giving shape `(H, W, 3)`. During training we rarely process one example at a time; we stack many into a **batch**, adding yet another axis. That is why image models work on 4-D tensors, typically `(N, C, H, W)` in PyTorch (batch, channels, height, width).

The pattern repeats everywhere: a sentence becomes `(batch, tokens, features)`; an audio clip becomes `(batch, samples)`; a table of records becomes `(rows, columns)`. Learning to read a shape tells you what each number means.

4 Anatomy of a tensor

Beyond its values, every tensor carries four pieces of metadata you will constantly refer to.



Figure 3. The four attributes you check most often: shape, rank, dtype and device (plus element count).

Shape and **rank** describe the arrangement of the data. **dtype** (data type) fixes how each element is stored — `float32` is the default for training, while `float16` / `bfloat16` save memory and `int64` is common for indices and labels. The **device** records *where* the tensor physically lives: in CPU RAM (`cpu`) or in GPU memory (`cuda:0`). This last attribute is where tensors and CUDA first meet — an operation can only run on the GPU if its tensors are already on the GPU.

5 How tensors live in memory

A tensor's neat grid is a convenient illusion. Underneath, the numbers sit in one long, flat block of memory.

Logical view: a 3 x 4 matrix

1	2	3	4
5	6	7	8
9	10	11	12



Physical storage: one contiguous, row-major (C-order) block

1	2	3	4	5	6	7	8	9	10	11	12
0	1	2	3	4	5	6	7	8	9	10	11

index

stride = (4, 1): move 1 row = +4 in memory, move 1 column = +1
A 'view' (reshape / transpose / slice) reuses this block - no copy.

Figure 4. A 2-D tensor and its flat, row-major storage. Strides translate multi-dimensional indices into a single memory offset.

Because the data is stored linearly, the tensor keeps a small tuple of **strides** that says how far to jump in memory for a one-step move along each axis. For the row-major matrix above, the strides are `(4, 1)`: stepping to the next row skips 4 elements, stepping to the next column skips 1.

This design is why many operations are almost free. A **view** — such as `reshape`, `transpose`, or a basic slice — usually just returns a new tensor with different shape and strides *pointing at the same memory*, copying nothing. A tensor whose elements are laid out in standard order is called **contiguous**; some operations require this, which is why you occasionally see a `.contiguous()` call to force a compact copy.

Why it matters for speed: contiguous, well-aligned memory lets the GPU read data in large efficient bursts. Scattered access patterns waste memory bandwidth, which is often the real bottleneck — not raw compute.

6 Broadcasting

Broadcasting lets tensors of different but compatible shapes combine in one operation, without you writing loops or manually copying data.

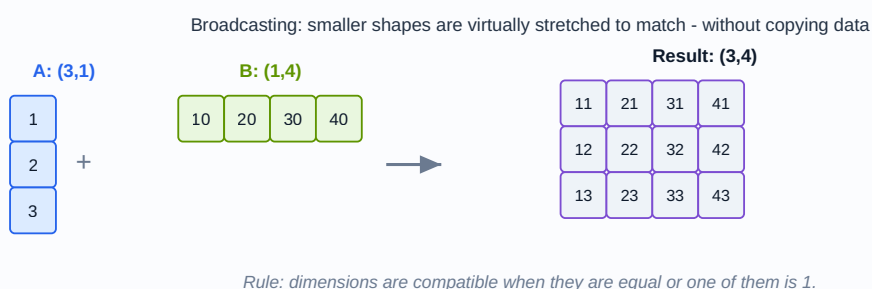


Figure 5. Broadcasting a (3,1) column and a (1,4) row into a (3,4) result by virtually stretching each along its size-1 axis.

The rule is simple: line the shapes up from the right, and two dimensions are compatible when they are equal or one of them is 1. A size-1 axis is *virtually* repeated to match the other operand — virtually, because the framework does not actually allocate the expanded data. This is what makes an expression like adding a bias vector to a whole batch both concise and efficient.

7 Core tensor operations

A handful of operation families cover most of deep learning. The star of the show is the matrix multiply.

- **Elementwise** — add, multiply, activations like ReLU; each output depends on one input position.
- **Reductions** — sum, mean, max over one or more axes; many outputs collapse to fewer.
- **Matrix multiply (matmul / GEMM)** — the core of linear layers, attention and convolutions.
- **Shape ops** — reshape, transpose, concatenate, slice; usually cheap view operations.

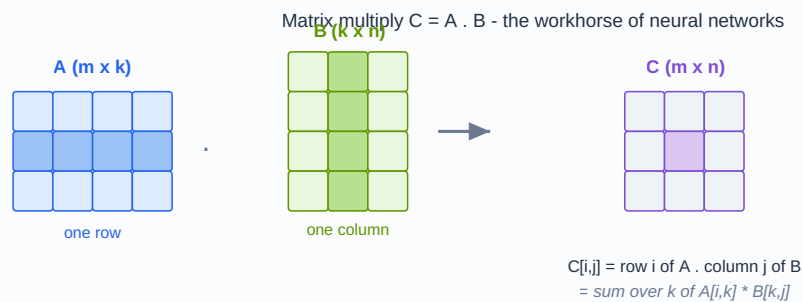


Figure 6. Matrix multiplication: every output element is a dot product of a row of A with a column of B.

Matrix multiply deserves special attention because it dominates the compute in most models and because it is exactly the kind of regular, repetitive arithmetic a GPU devours. Each of the many output elements is an independent dot product, so thousands can be computed at the same time — a perfect match for parallel hardware, as the later sections show.

8 Tensors in PyTorch

PyTorch's central object is `torch.Tensor`. It feels like NumPy, but adds two superpowers: it can live on the GPU, and it can track gradients automatically.

```
import torch

# create tensors
x = torch.tensor([[1., 2.], [3., 4.]]) # from data, shape (2,2)
z = torch.zeros(2, 3)                 # filled with zeros
r = torch.randn(4, 4, device="cuda")  # created directly on the GPU

print(x.shape, x.dtype, x.device)      # torch.Size([2, 2]) torch.float32 cpu

# move an existing tensor to the GPU
x = x.to("cuda")                      # now x.device == cuda:0
```

PyTorch runs **eagerly** (define-by-run): each line executes immediately, which makes debugging feel like ordinary Python. Its **autograd** engine records the operations you perform into a graph on the fly, then walks it backwards when you call `.backward()` to compute every gradient via the chain rule.

```
# autograd in action
w = torch.randn(3, requires_grad=True) # track operations on w
y = (w * 2).sum()                      # build the graph as you compute
y.backward()                           # fill in gradients
print(w.grad)                          # tensor([2., 2., 2.] )
```

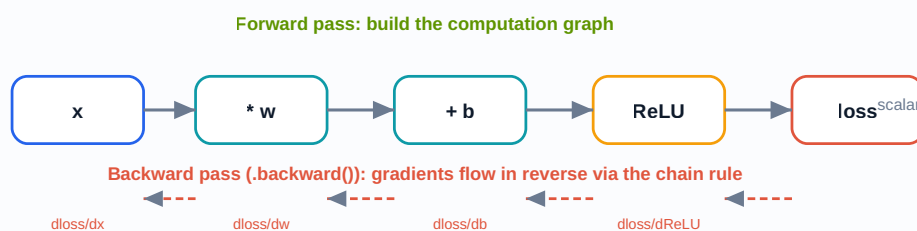


Figure 7. Autograd records a forward graph, then propagates gradients backward through it when you call `.backward()`.

9 Tensors in TensorFlow

TensorFlow's tensor is `tf.Tensor`. A key difference: a `tf.Tensor` is **immutable**. Trainable, updatable state is held separately in a `tf.Variable`.


```
import tensorflow as tf

# tf.Tensor is immutable; tf.Variable holds trainable state
x = tf.constant([[1., 2.], [3., 4.]]) # shape (2,2), dtype float32
w = tf.Variable(tf.random.normal([3])) # a trainable variable

# TensorFlow places ops on a visible GPU automatically
with tf.device("/GPU:0"):
    y = tf.matmul(x, x) # runs on the GPU

# automatic differentiation with GradientTape
with tf.GradientTape() as tape:
    loss = tf.reduce_sum(w * 2)
grad = tape.gradient(loss, w) # tensor([2., 2., 2.]
```

Modern TensorFlow also runs eagerly by default. For speed and deployment you can wrap a function in `@tf.function`, which traces it into a static graph that can be optimised and run without Python overhead. Where PyTorch asks you to record gradients with `requires_grad`, TensorFlow asks you to record them inside a `tf.GradientTape` context.

10 PyTorch vs TensorFlow at a glance

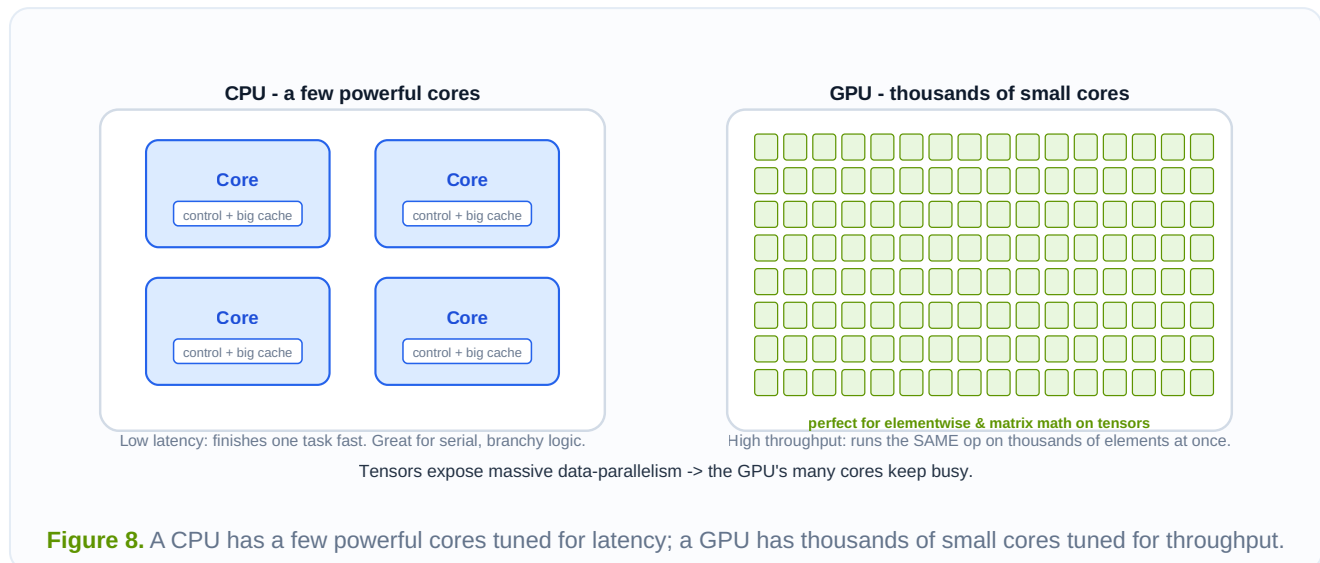
Both frameworks compute on the same kind of tensors and, underneath, call the same CUDA libraries. They differ mostly in style and ecosystem.

Aspect	PyTorch	TensorFlow
Core tensor object	<code>torch.Tensor</code> (mutable)	<code>tf.Tensor</code> (immutable) + <code>tf.Variable</code>
Execution model	Eager by default; compile with <code>torch.compile</code>	Eager by default; graph with <code>@tf.function</code>
Automatic gradients	<code>requires_grad</code> + <code>.backward()</code>	<code>tf.GradientTape</code>
Send data to GPU	Explicit: <code>.to("cuda")</code>	Automatic placement; <code>tf.device()</code> to control
High-level API	<code>torch.nn</code> , Lightning	Keras
Typical strength	Research, fast iteration, flexibility	Production & deployment (TF Serving, TF Lite)

The important point: whichever you pick, a linear layer's matrix multiply ends up in the same NVIDIA library (cuBLAS) running on the same GPU. The framework is the friendly wrapper; CUDA does the heavy lifting.

11 Why GPUs are built for tensors

A CPU is optimised to finish one complicated task quickly. A GPU is optimised to finish a huge number of simple, identical tasks together. Tensor math is exactly the second kind of workload.



Consider adding two vectors with a million elements. On a CPU you largely loop through them (helped by a few cores and vector instructions). On a GPU, a million tiny threads can each add one pair of elements at essentially the same time. Because tensor operations apply the *same* arithmetic to *every* element independently, they expose exactly the kind of **data parallelism** a GPU is designed to exploit. This execution style — one instruction driving many data lanes — is called **SIMT** (Single Instruction, Multiple Threads).

12 CUDA in a nutshell

CUDA is NVIDIA's platform for general-purpose computing on GPUs. It lets programs launch functions called **kernels** that run across thousands of GPU threads. Frameworks generate and launch these kernels for you.

A kernel is written once, from the point of view of a single thread, and then launched over a whole grid of them. Here is the classic vector-add kernel — the GPU version of the elementwise operation from earlier:

```
// A CUDA kernel: each thread handles ONE element
__global__ void add(const float* A, const float* B, float* C, int n) {
    int i = blockIdx.x * blockDim.x + threadIdx.x; // this thread's global index
    if (i < n) C[i] = A[i] + B[i];                // no loop – one add per thread
}
```

Threads are organised in a three-level hierarchy. Many threads form a **block**; many blocks form a **grid**; and the hardware runs threads in groups of 32 called **warps**, which execute in lock-step.

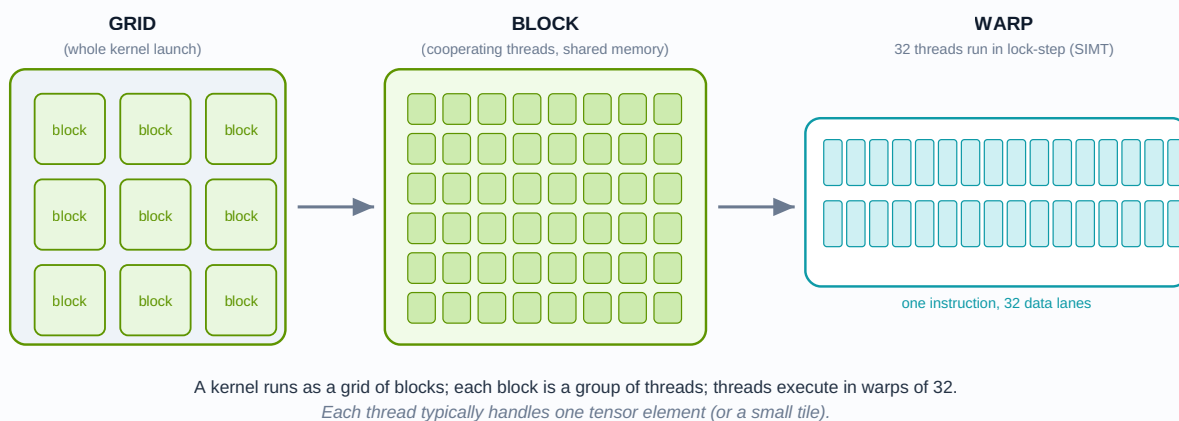


Figure 9. CUDA's execution hierarchy: a grid of blocks, each block a group of cooperating threads, executed in warps of 32.

Just as important as threads is **memory**. The GPU has a steep memory hierarchy, and fast kernels are usually the ones that keep data in the small, fast on-chip memories instead of repeatedly reaching out to large, slow global memory (HBM).

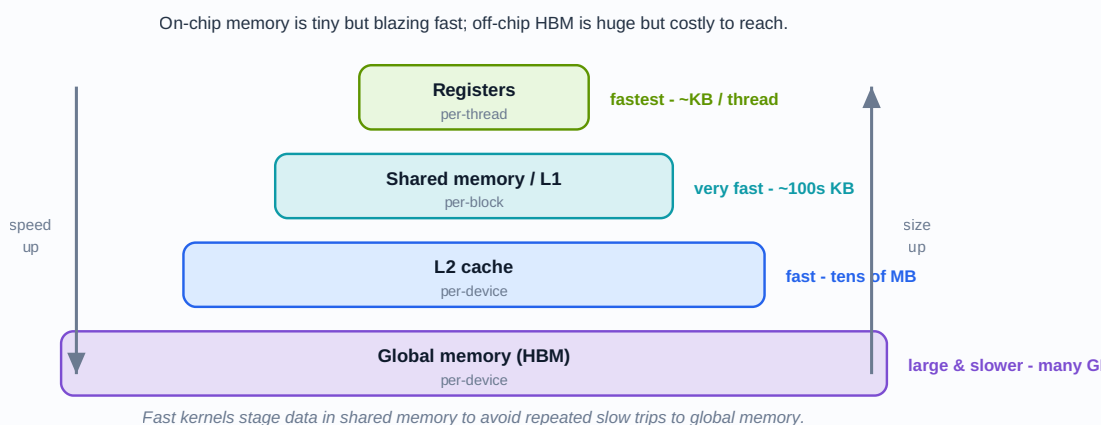


Figure 10. The GPU memory hierarchy. On-chip registers and shared memory are tiny but extremely fast; global HBM is huge but slower.

A physical GPU is built from many **Streaming Multiprocessors (SMs)**. Each SM runs many threads, owns a pool of registers and a slab of shared memory, and contains the arithmetic units — including the specialised Tensor Cores discussed next.

13 How CUDA accelerates tensor math

Now we connect the two halves of the guide: mapping the tensor operations from Part 7 onto the CUDA execution model from Part 12.

Elementwise ops: one thread per element

Elementwise operations are the easy, perfect case. The framework launches one thread per output element, and the whole tensor is processed in a single parallel sweep — no loop in sight.

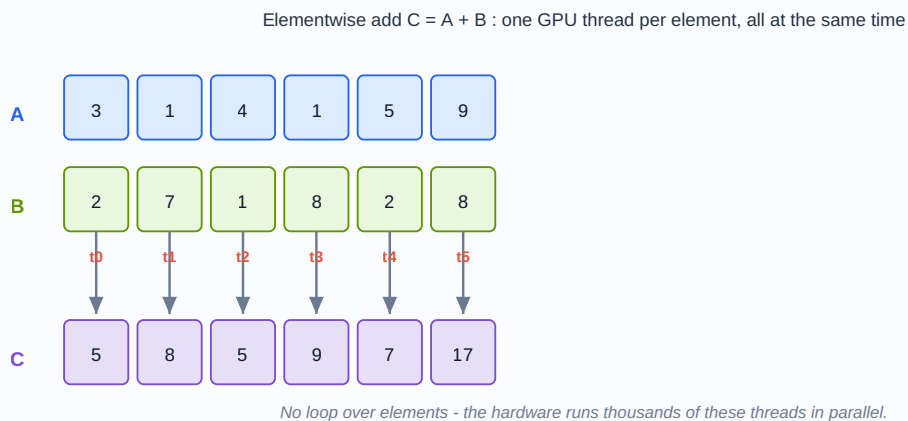


Figure 11. An elementwise add mapped onto GPU threads: thread t_i computes $C[i] = A[i] + B[i]$, all in parallel.

Matrix multiply: tiling with shared memory

Matrix multiply reuses each input value many times, so a naive kernel would fetch the same numbers from slow global memory over and over. High-performance GEMM kernels instead **tile** the matrices: each block loads small tiles of A and B into fast shared memory once, and every thread in the block reuses them to build a tile of the output.

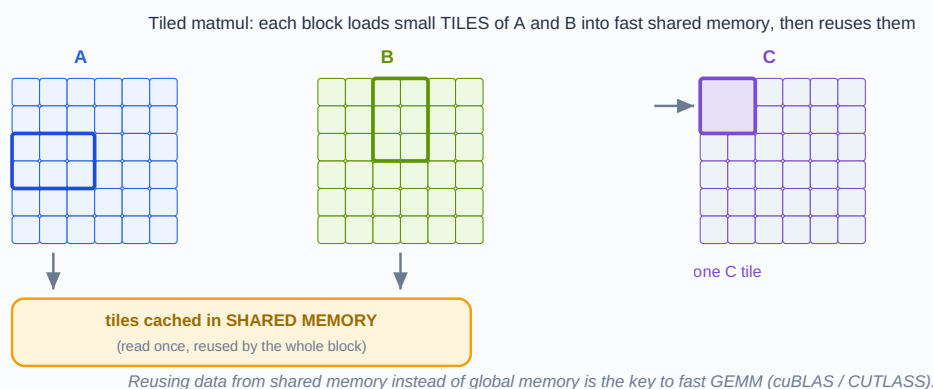


Figure 12. Tiled matrix multiply. Loading tiles into shared memory turns many slow global reads into a few, which is the essence of a fast GEMM.

You rarely write these kernels yourself

NVIDIA ships hand-tuned libraries that the frameworks call automatically: **cuBLAS** and **CUTLASS** for matrix multiplies, and **cuDNN** for deep-learning primitives such as convolutions, attention and normalisation. When

you write `torch.matmul` or a Keras `Dense` layer, one of these highly optimised kernels does the actual work.

Tensor Cores and mixed precision

Recent GPUs add **Tensor Cores**: special units that perform a small matrix-multiply-and-accumulate in a single step, typically taking lower-precision inputs (FP16/BF16) while accumulating in higher-precision FP32 to preserve accuracy. This is the hardware behind **mixed-precision training**, which can cut memory use and multiply throughput with little effect on final model quality.

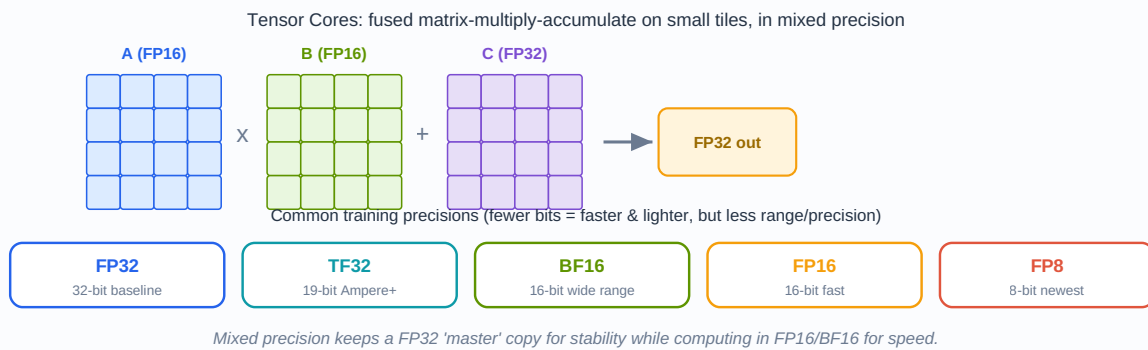


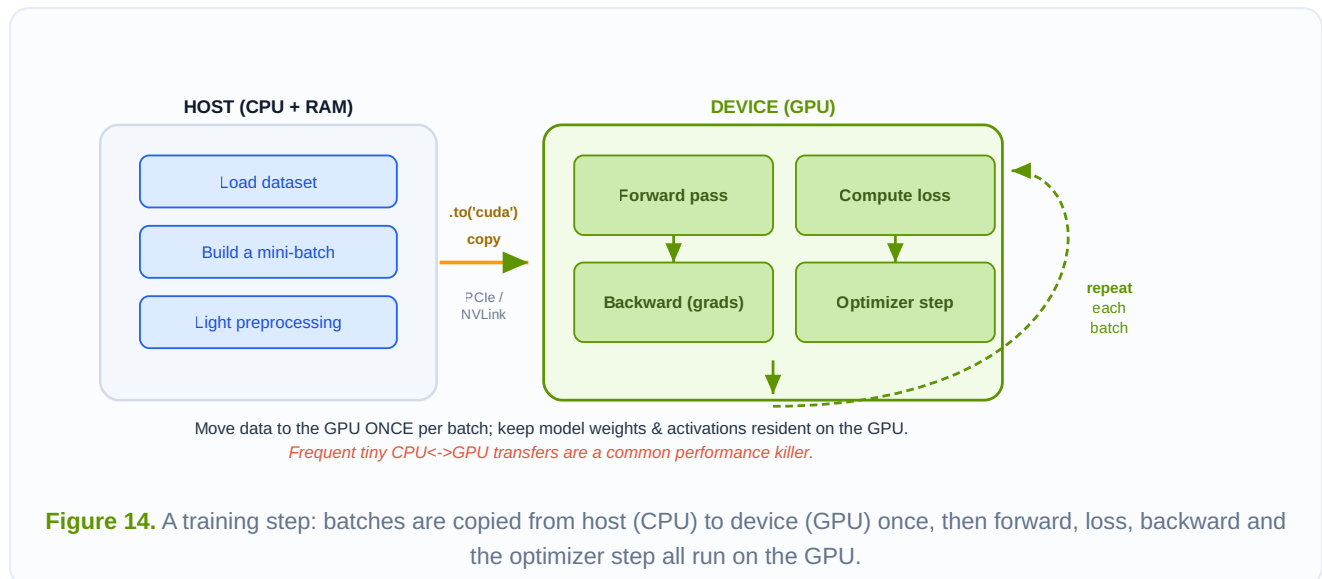
Figure 13. Tensor Cores fuse a small matrix multiply-accumulate, taking low-precision inputs and accumulating in FP32. Fewer bits mean more speed and less memory.

```
# PyTorch mixed precision: a few lines around the normal loop
scaler = torch.cuda.amp.GradScaler()
for x, y in loader:
    x, y = x.cuda(), y.cuda()
    with torch.autocast("cuda", dtype=torch.float16): # run in FP16 on Tensor Cores
        loss = loss_fn(model(x), y)
    scaler.scale(loss).backward() # scale to protect small grads
    scaler.step(opt); scaler.update(); opt.zero_grad()
```

Takeaway: CUDA speeds up tensor learning on three levels at once — massive thread parallelism for every op, a memory hierarchy that clever kernels exploit, and dedicated matrix hardware (Tensor Cores) for the multiplies that dominate training.

14 Data movement & the GPU training loop

Raw speed is wasted if the GPU sits idle waiting for data. Where tensors live, and how often they cross between CPU and GPU, can matter as much as the maths.



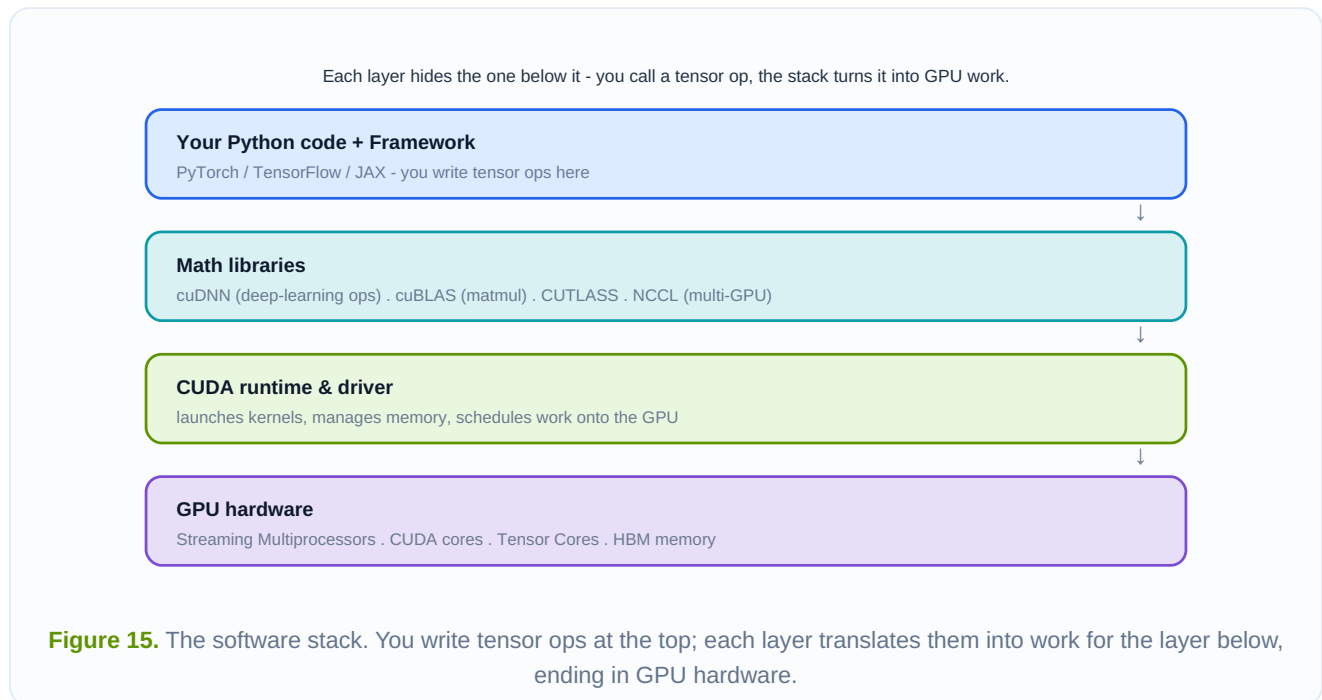
The CPU (the **host**) and GPU (the **device**) have separate memories connected by a link such as PCIe or NVLink. Copying a tensor across that link costs time. The winning pattern is to move each batch to the GPU *once*, keep the model's weights and activations resident on the GPU throughout, and only bring small results (like a loss value) back to the CPU when you actually need to read them.

Common pitfalls:

- **Device mismatch** — an operation between a `cpu` tensor and a `cuda` tensor throws an error. Keep operands on the same device.
- **Transfer thrashing** — calling `.cpu()` / `.item()` or printing GPU tensors inside a tight loop forces synchronisation and slow copies.
- **Starved GPU** — slow data loading or preprocessing on the CPU can leave the GPU waiting; overlap loading with compute (e.g. multiple worker processes, pinned memory).

15 The full stack, and key takeaways

Putting it together: a single line of framework code descends through several layers before it becomes electricity moving in the GPU.



When you call `torch.matmul(a, b)`, PyTorch checks the tensors' device and dtype, dispatches to cuBLAS, which selects a tuned kernel, which the CUDA runtime launches across the GPU's SMs and Tensor Cores, reading and writing HBM memory. You get a result tensor back — the whole descent hidden behind one function call.

Key takeaways

Idea	In one sentence
Tensor	An N-dimensional array of numbers, described by shape, rank, dtype and device.
Frameworks	PyTorch and TensorFlow both operate on tensors and add automatic differentiation.
Views & memory	Reshapes and slices usually share memory; contiguous layout keeps the GPU fed.
Why GPUs	Tensor ops are data-parallel, matching the GPU's thousands of cores.
CUDA model	Kernels run over a grid of blocks of threads, executed in warps of 32.
Fast kernels	Reuse data from fast on-chip memory; call tuned libraries (cuBLAS, cuDNN).
Tensor Cores	Dedicated matrix hardware plus mixed precision multiply training throughput.
Data movement	Keep tensors on the GPU; avoid needless CPU ↔ GPU transfers.

16 Glossary

Term	Meaning
Rank / ndim	The number of axes (dimensions) of a tensor.
Shape	Tuple of sizes along each axis, e.g. (N, C, H, W).
dtype	The numeric type of each element (float32, float16, int64, ...).
Device	Where a tensor lives: CPU memory or a specific GPU (cuda:0).
Stride	How many elements to skip in memory to move one step along an axis.
Contiguous	Elements laid out in standard row-major order with no gaps.
Broadcasting	Combining compatible shapes by virtually stretching size-1 axes.
Autograd / autodiff	Automatic computation of gradients by recording the forward operations.
Kernel	A function that runs on the GPU across many threads.
Thread / block / grid	CUDA's three-level hierarchy of parallel work.
Warp	A group of 32 threads that execute the same instruction together (SIMT).
SM	Streaming Multiprocessor — a hardware unit of the GPU that runs blocks.
Shared memory	Fast on-chip memory shared by the threads of a block.
Global memory (HBM)	The GPU's large main memory; high bandwidth but higher latency.
cuBLAS / cuDNN	NVIDIA libraries of tuned kernels for matrix math and deep-learning ops.
Tensor Core	Specialised unit performing a fused matrix multiply-accumulate.
Mixed precision	Computing in FP16/BF16 while keeping an FP32 master for stability.
Host / device	The CPU (host) and the GPU (device), each with its own memory.

Version landscape at time of writing (mid-2026): CUDA Toolkit 13.x, cuDNN 9.x, PyTorch 2.x (CUDA 12.8 / 13.x builds), TensorFlow 2.x. Exact numbers move quickly; the concepts in this guide are stable across versions.